

Analysis of Algorithms

* Algorithm

- ① An Algorithm is a meaningful set of instructions which is in a sequential manner.
- ② Algorithm should satisfy the following criteria
 - a) Input
 - b) Output
 - c) Definiteness
 - d) Finiteness
 - e) Effectiveness

- ③ Performance Analysis have 2 types
 1. Time Complexity
 2. Space Complexity

- ④
 - a) Time Complexity:
 - Best case
 - Worst case
 - Average case

How much time the program takes to execute
 - b) Space Complexity
 - Instruction Space
 - Data Space
 - Environment stack space

How much space/memory a program takes

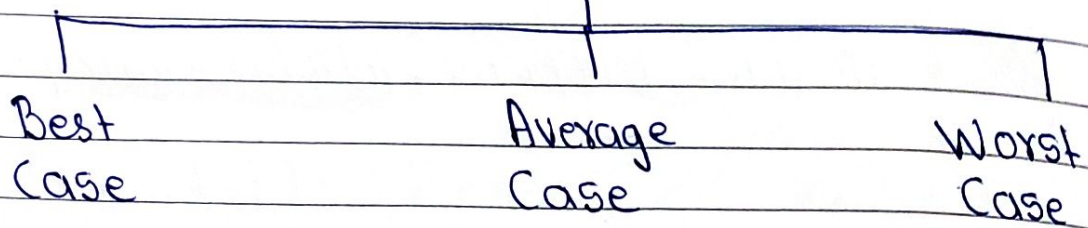
- ④ We can Compare Algorithms based on 3 criteria
 - a) Execution time
 - b) Number of statements executed
 - c) Running time Analysis

*we always compare two algo by its worst case

Page No.:

Date:

Time Complexity



Space Complexity



- ① Instruction Space: -
Store the compiled version of the program instructions.
- ② Data Space: -
Store all constant and variable values
- ③ Environment Stack space: -
Save information needed to resume execution of partially completed functions

There are two approaches to calculate Time

- Space Complexity: groups out to notification
- 1) Frequency count / Step count Method (Independent)
 - 2) Asymptotic Notations (dependent)

* Frequency Count / Step count Method Rules

- ① Ignore lower order exponents and also ignore the constant value of higher order exponent
- ② For comments, declaration
Count = 0
- ③ return and assignment statement
Count = 1

eg ignore constant $f(n) = 6n^3 + 10n^2 + 15n + 3$ ignore lower order exponents.

$$\text{So } f(n) = O(n^3)$$

eg Sum of n values of an array

① Algorithm sum(int a[], int n)

```

S = 0;
for(i = 0; i < n; i++)
{
    S = S + a[i];
}
return S;

```

Time Complexity	Space Complexity
$n+1$	$a[] = n \text{ words}$
n	$n = 1 \text{ word}$
1	$S = 1 \text{ word}$
	$i = 1 \text{ word}$
$2n+3$	$n+3$
$f(n) = O(n)$	$\approx O(n)$

2 Addition of two square Matrices of dimension

Algorithm: addMat (int a[][], int b[][])

```

{
    int c[][];
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
}

```

Time Complexity	Space Complexity
$n+1$ $n \times (n+1)$ $n \times n$	$a[i][j] = n^2 \text{ words}$ $b[i][j] = n^2 \text{ words}$ $c[i][j] = n^2 \text{ words}$
$n+1 + n^2 + n + n^2$ $2n^2 + 2n + 1$ $f(n) = O(n^2)$	$i = 1 \text{ word}$ $j = 1 \text{ word}$ $n = 1 \text{ word}$ $3n^2 + 3$ $O(n^2)$

loops (By Step-Count Method)

1. $\text{for}(i=0; i < n; i++)$ & $\text{for}(i=n; i > n; i--)$
 $\{$ $\{$
 Statements; Statements;
 $\}$ $\}$

* Time Complexity of Both

$\rightarrow n+1$

$\rightarrow n$

$$\therefore f(n) = 2n + 1$$

$$f(n) = O(n)$$

2. $\text{for}(i=1; i < n; i=i+2)$
 $\{$
 Statements;
 $\}$

* Time Complexity

$\rightarrow n+1$

$\rightarrow n/2$

$$\therefore f(n) = 3n/2 + 1$$

$$f(n) = O(n)$$


```

3. for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            Statements;
        }
    }

```

*

Time Complexity

$\rightarrow n+1$
 $\rightarrow n(n+1)$
 $\rightarrow n \times n$

$$f(n) = 2n^2 + 2n + 1$$

$$f(n) = O(n^2)$$

Time Complexity

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$f(n) = O(n^2)$$

loops (By tracing)

```

① for(i=0; i<n; i++)
{
    for(j=0; j<i; j++)
    {
        Statements;
    }
}

```

Time Complexity

i	j	Statements
0	0	0
1	0	1
2	0, 1	2
3	0, 1, 2	3
...	...	...
N	0 to n-1	n

$$1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$$

$$T(n) = 1 + 2 + 3 + 4 + \dots + n + 1 = \frac{(n+1)(n)}{2}$$

$$= O(n^2)$$

② $p = 0$
 for($i = 1; p \leq n; i++$)
 {
 $p = p + i;$
 }

($p = 0$) equal
 ($1 + 2 + \dots + i = 0 = i$) not
 ($1 + 2 + \dots + i = 0 = i$) not
 not a statement

A

Time Complexity	
i	statements
1	1
2	1
3	1
4	1
5	1
6	1
k	1

$$= 1 + 2 + 3 + 4 + \dots + k > n$$

$$= k(k+1)/2 > n$$

$$= k^2 + k/2 > n$$

$$\approx k^2 > n$$

$$k = \sqrt{n}$$

$$O(\sqrt{n}) = 1 + 2 + \dots + 4 + 5 + 6 + 7 + 8 + 9 + 10 + \dots$$

$$O(1 + 2 + \dots + 10) = 1 + 2 + \dots + 10 + 11 + 12 + 13 + 14 + 15 + 16 + 17 + 18 + 19 + 20 = (20) T$$

$$O(n) =$$

③ for($i=1; i < n; i=i*2$)

{ statements; }

}

Time Complexity

i	Statements
$1 * 2^0$	1
$1 * 2$	1
$1 * 2 * 2$	1
$1 * 2 * 2 * 2$	1
...	...
2^k	1

$$i > n$$

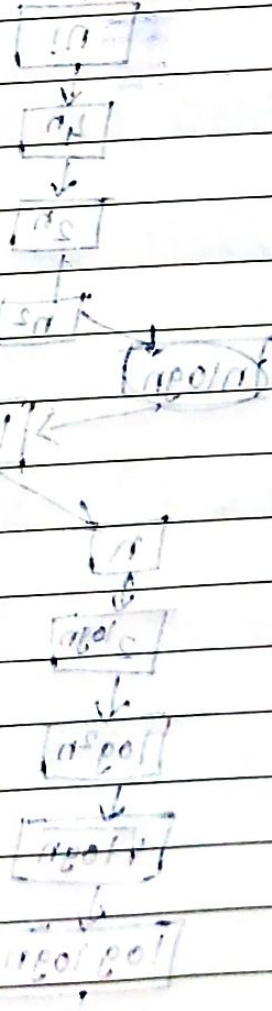
$$i > 2^k$$

$$2^k > n$$

$$2^k = n$$

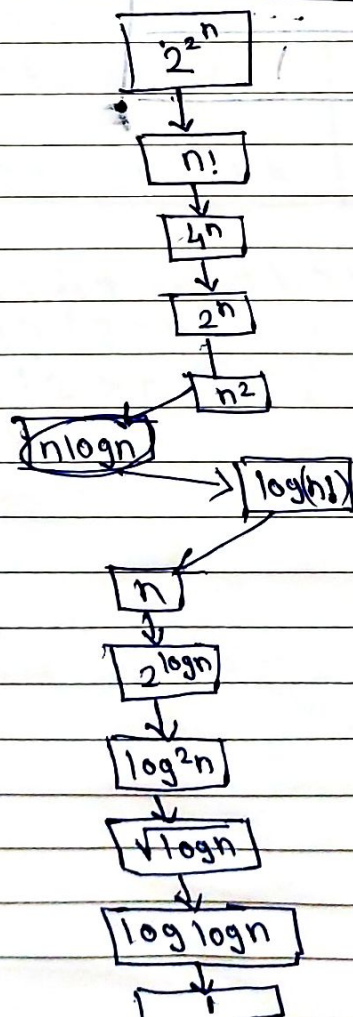
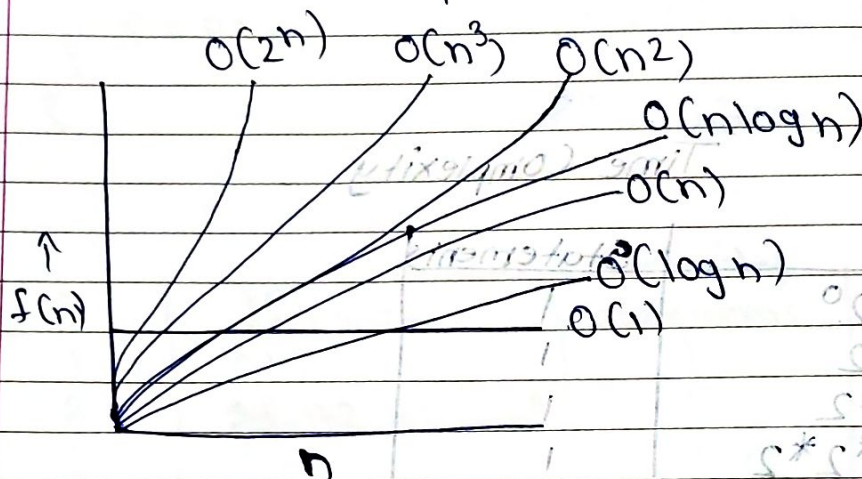
$$k = \log_2 n$$

$$= O(\log_2 n)$$



* Rate of Growth

Rate at which the running time increases as a function of input



Decreasing
Rate of
growth.

(log log) 0 =

Big - OH (o) ~~Big - Omega (o)~~

① $f(n) = 3n + 8$

$g(n) = 4n$ $C = 4$

$$f(n) \leq Cg(n)$$

$$3n + 8 \leq 4n$$

$$\therefore n = 6$$

$$= 3(6) + 8 \leq 4(6)$$

$$= 26 \leq 24$$

② $f(n) = 3n + 8$

$$g(n) = 5n$$

$$3n + 8 = 5n$$

$$8 = 2n$$

$$n = 4$$

③ $f(n) = 3n + 8$

$$g(n) = 5n$$

~~$$f(n) = n^3$$~~
~~$$g(n) = 2n^2$$~~

no



(4) $f(n) = n^2 + 1$ (2) $f(n) = O(g(n))$
 $g(n) = 2n^2$

$$f(n) \leq g(n)$$

$$n^2 + 1 \leq 2n^2$$

$$1 \leq n^2$$

$$n \geq 1$$

$$n \geq 1$$

$$n_0 = 1$$

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)$$

$$f(n) \leq c \cdot g(n)$$

$$c = 1$$

(5) $f(n) = \frac{3}{2}n^2 + 1$

$$f(n) = O(g(n))$$

let $g(n) = 2n^2$

(3) $f(n) = O(g(n))$

$$\frac{3n^2}{2} + 1 = 2n^2$$

$$3 = 1.5$$

$$1 + n^2 + n^2 = (n)^2$$

$$2n^2 - 1.5n^2$$

$$f(n) = O(g(n))$$

$$1 = \frac{1}{2}n^2$$

$$1 = n^2$$

$$n^2 = 2$$

$$n = \sqrt{2}$$

$$f(n) \leq 1 + n^2 + n^2 \leq 2n^2$$

$$c = 2$$

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)$$

$$c = 2$$

$$f(n) \leq c \cdot g(n)$$

Big Omega(Ω)

$$1 + \frac{1}{n} = (n)^{-1}$$

$$\frac{1}{n} = (n)^{-1}$$

$$f(n) = 5n^2$$

$$c g(n) \leq f(n)$$

$$5n^2 \leq 5n^2$$

$$n = 1 = n_0$$

$$f(n) = \Omega(g(n))$$

$$1 + \frac{1}{n} = (n)^{-1} + \frac{1}{n}$$

$$\frac{1}{n} = (n)^{-1}$$

Big Theta(Θ)

$$f(n) = 4n^2 + 4n + 1$$

$$g(n) = n^2$$

$$\frac{1}{n} = 1 + \frac{1}{n}$$

$$\frac{1}{n} = 1 + \frac{1}{n}$$

$$4n^2 \leq 4n^2 + 4n + 1 \leq 5n^2 \quad \therefore n = 1$$

$$9 \neq 5$$

$$\frac{1}{n} = \frac{1}{n}$$

$$\frac{1}{n} = \frac{1}{n}$$

$$\therefore n = 2$$

$$16 \leq 25 \leq 20$$

$$n = 3$$

$$36 \leq 49 \leq 45$$

Substitution Method

★

Two-type of Substitution

a) forward - Best to n Case.

b) Backward

★ Backward Substitution (Factorial)

Q
$$T(n) = T(n-1) + C \quad n > 1$$

$$= d \quad n = 1$$

i) $T(1) = d$

ii) $T(2) = T(2-1) + C$

$$= T(1) + C = d + C$$

iii) $T(3) = T(3-1) + C$

$$= T(2) + C = d + 2C$$

iv) $T(n-2) = T(n-2-1) + C$

$$= T(n-3) + C$$

$$= d + T(n-3)C$$

v) $T(n) = d + (n-1)C$

$$= \Theta(n)$$

★ Backward Substitution (Factorial)

Q
$$T(n) = T(n-1) + n \quad n > 1$$

$$= 1 \quad n = 1$$

$$T(n-1) = T(n-1-1) + n-1$$

$$= T(n-2) + (n-1)$$

$$T(n) = T(n-2) + n + (n-1) \quad \text{--- (2)}$$

$$T(n-2) = T(n-2-1) + n-2$$

$$T(n-3) + n-2$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad (3)$$

It will go till k^{th} iteration

$$T(n) = T(n-k) + (n-(k-1)) + (n-(k-2)) + \dots$$

$$n-k=1 \quad k=n-1$$

$$\begin{aligned} T(n) &= T(1) + (n-1-n+1) + (n-2-(n-1-1)) + \dots + 1 \\ &= T(1) + 2 + 3 + 4 + \dots + n \\ &= 1 + 2 + 3 + \dots + n \\ &= n(n+1) \end{aligned}$$

$$T(n) = \Theta^2(n^2)$$

$$T(n) = n + 2T\left(\frac{n}{2}\right) \quad n > 1$$

$$n=1$$

$$T\left(\frac{n}{2}\right) = \frac{n}{2} + 2T\left(\frac{n}{4}\right)$$

$$T(n) = n + 2 \left[2T\left(\frac{n}{4}\right) + \frac{n}{2} \right]$$

$$T(n) = n + 4T\left(\frac{n}{4}\right) + n$$

$$= 4T\left(\frac{n}{4}\right) + 2n \quad \text{or } 2^2 T\left(\frac{n}{2^2}\right) + 2n$$

$$= 2^{\frac{2}{2}} T\left(\frac{n}{4}\right) = \frac{n}{4} + 2T\left(\frac{n}{8}\right)$$

$$T(n) = 4\left[\frac{n}{4} + 2T\left(\frac{n}{8}\right)\right] + 2n$$

$$= n + 8T\left(\frac{n}{8}\right) + 2n$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3n$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

$$= nT(1) + n \log n$$

$$= n + n \log n$$

$$= \Theta(n \log n)$$

$$\frac{n}{2^k} = 1 \Rightarrow k = \log n$$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$f(n) = O(n^k)$$

$$1) a < 1$$

$$T(n) = O(n^k)$$

$$2) a = 1$$

$$T(n) = O(n^{k+1})$$

$$3) a > 1$$

$$T(n) = O(n^k a^{n/b})$$

$$(1) T(n) = T(n-1) + c$$

$$\therefore f(n) = O(n)$$

$$T(n) = O(n^{0+1})$$

$$= O(n)$$

$$\therefore n^k = 1$$

$$k = 0$$

$$(2) T(n) = T(n-1) + n$$

$$\therefore a = 2 > 1$$

$$n^k = n^1$$

$$k = 1$$

$$T(n) = O(n^k a^{n/b})$$

$$= O(n \cdot 2^n)$$

$$\neq O(n)$$

$$k = 1$$

$$\begin{aligned} T(n) &= O(n^k \cdot a^{n/b}) \\ &= O(3^{n/1}) = O(3^n) \\ &= O(3^n) \end{aligned}$$

$$f(n) = O(n^2)$$
$$n^k = n^2$$
$$k = 2$$

$$T(n) = O(n^2 + 1)$$

$$= O(n^3)$$

$$T(n) = T(n-k) + \log n + \log(n-1) + \log(n-2) + \dots + \log(n-k+1)$$

$$1 < c < \infty$$

$$n - k = 0$$

$$n = k$$

$$0 < n$$

$$0 = n$$

$$\therefore \log a + \log b = \log ab$$

$$\begin{aligned} T(n) &= T(n/2 + \log n) + \log(n-1) + \log(n-2) \\ &\quad + \dots + \log(1) \\ &= 1 + \log(n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 3 \cdot 2 \cdot 1) \\ &= 1 + \log(n!) \end{aligned}$$

$$\therefore T(n) = O(\log n^n) = O(n \log n)$$

$$T(n) = T(\sqrt{n}) + \log n \quad \text{--- (1)}$$

$$\text{Let } n = 2^m$$

$$m = \log_2 n$$

$$T(2^m) = T(2^{m/2}) + m \quad \text{--- (2)}$$

$$S(m) = T(2^m)$$

$$S(m) = 2S\left(\frac{m}{2}\right) + m \quad \text{--- (3)}$$

$$S(m) = 2S\left(\frac{m}{2}\right) + m$$

$$a=2, b=2, k=1, p=0$$

$$S(m) = O(m^k \log^{p+1} m)$$

$$S(m) = O(m \cdot \log m)$$

$$T(n)$$

$$S(m) = T(2^m)$$

$$n = 2^m$$

$$m = \log n$$

$$T(n) = \Theta(\log n \cdot \log \log n)$$

Min-Max

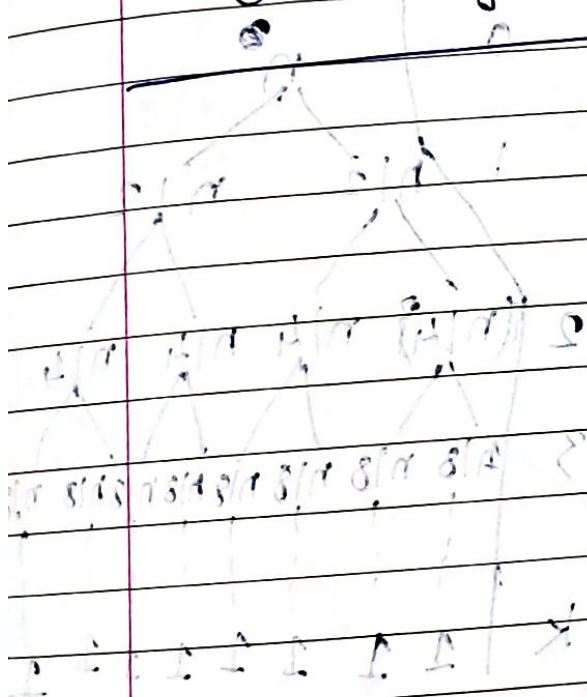
$$T(n) = 2T\left(\frac{n}{2}\right) + 2$$

$$n > 2$$

$$n = 2$$

$$= 2$$

$$\Theta(n \cdot \log n)$$



$$T(n) = \Theta(n \cdot \log n)$$

$$n = \frac{n}{2}$$

$$n = 2^k \Rightarrow k = \log n$$

$$S = 200$$

$$+ 200$$

$$10 \cdot 10$$

$$n = 400$$

$$n$$

$$1$$

$$n$$

$$2$$

$$n$$

$$4$$

$$n$$

$$8$$

$$n$$

$$16$$

$$n$$

$$32$$

$$n$$

$$64$$

Recurrence Tree Method

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$

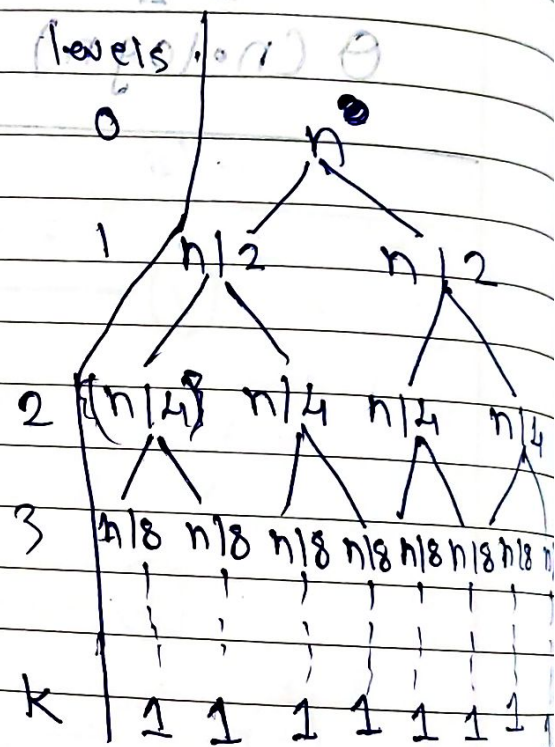
$\rightarrow$ No. of subproblems. $\leftarrow$ Cost incurred for dividing & combining

$\downarrow$
 $s =$ size of subproblem

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$\frac{n}{2^k} = 1$$

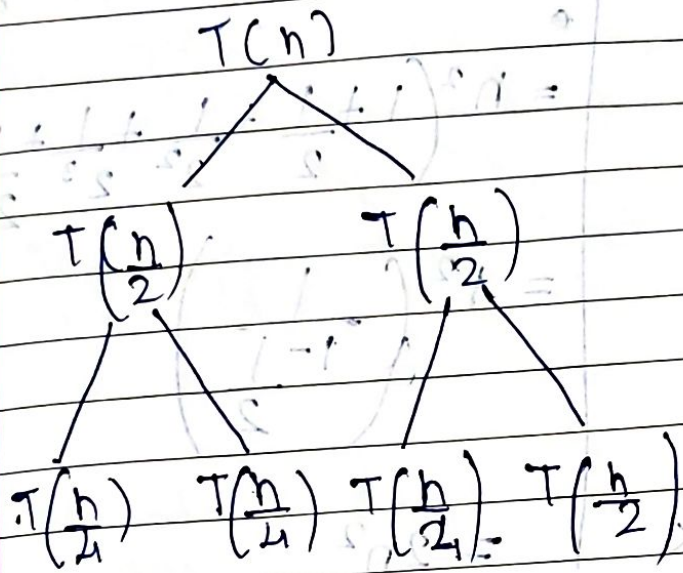
$$n = 2^k \Rightarrow k = \log n$$



no. of nodes.	Cost	pos = 2
1	n	Total Cost = n^k $= n \times \log n$
2	n	
4	n	
8	n	
$\vdots$	$\vdots$	
$n = 2^k$	n	$T(n) = \Theta(n \log n)$
	n^k	

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

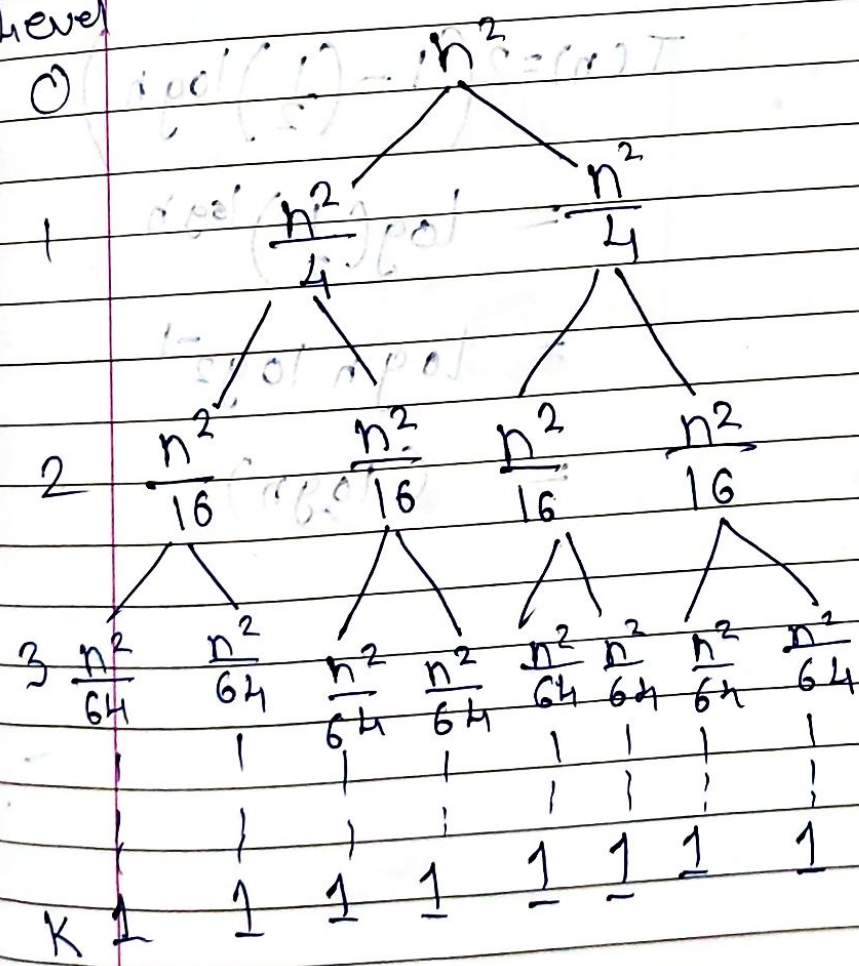


$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \left(\frac{n}{2}\right)^2$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \left(\frac{n}{4}\right)^2$$

$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{16}\right) + \left(\frac{n}{8}\right)^2$$

Level



no. of nodes.	cost	Total Cost
1	n^2	$n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \frac{n^2}{8} + \dots$
2	$\frac{n^2}{2}$	$= n^2 \left(1 + \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \dots \right)$
4	$\frac{n^2}{4}$	$= n^2 \left(\frac{1}{1 - \frac{1}{2}} \right)$
8	$\frac{n^2}{8}$	
$n = 2^k$	$\frac{n^2}{2^k}$	$= 2n^2$

$$T(n) = \Theta(n^2)$$

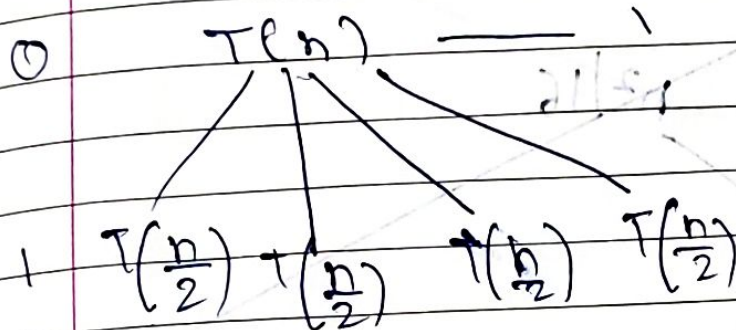
$$T(n) = 2 \left(1 - \left(\frac{1}{2} \right) \log n \right)$$

$$= \log \left(\frac{1}{2} \right) \log n$$

$$= \log n \log 2^{-1}$$

$$= (\log n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$



2

3

no. of
nodes
1

Cost

 n^2 Total cost = kn^2 $T(n) = \Theta(n^2 \log n)$ $n^2/2$ $n^2/4$

4k

 ~~kn^2~~ ~~$\log n$~~ $\frac{n}{2k} = 1$ $n = 2k$ $k = \log n$

$$n=1$$
$$n > 1$$

$$T\left(\frac{n}{2}\right)$$
~~n2/16~~

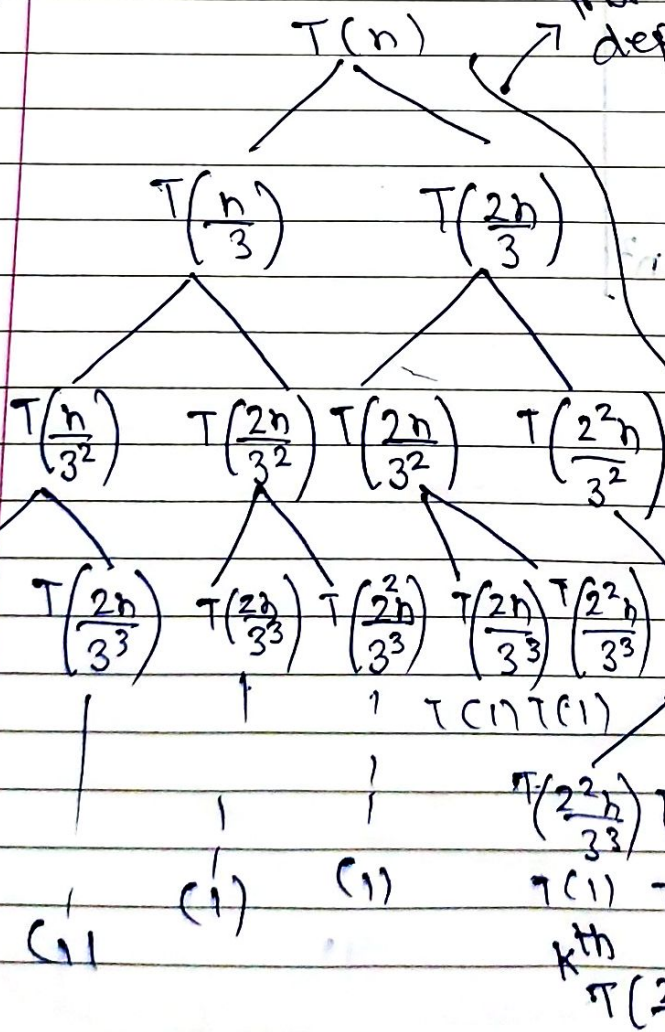
~~$$T(n) = \frac{n^2}{4^2} = \frac{n^2}{16}$$~~

Faint

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n \quad n > 1$$

○

1



maximum
depth.

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{n}{3^2}\right) + T\left(\frac{2n}{3^2}\right) + \frac{n}{3}$$

$$T\left(\frac{2n}{3}\right) = T\left(\frac{2n}{3^2}\right) + T\left(\frac{2^2 n}{3^2}\right) + \frac{2n}{3}$$

$$\bar{T}\left(\frac{n}{3^2}\right) = T\left(\frac{n}{3^2}\right) + T\left(\frac{2n}{3^3}\right) + \frac{n}{3^2}$$

$$T\left(\frac{2n}{3^2}\right) = T\left(\frac{2n}{81}\right) + T\left(\frac{2n}{3^3}\right) + \frac{2n}{3^2}$$

$$T\left(\frac{2^2 n}{3^2}\right) = T\left(\frac{2^2 n}{3^3}\right) + T\left(\frac{2^2 n}{3^2}\right) + \frac{2n}{3^2}$$

217

51

(ii)

٢٢

 k^{th}

$$T\left(\frac{2K}{3}\right)^L n$$

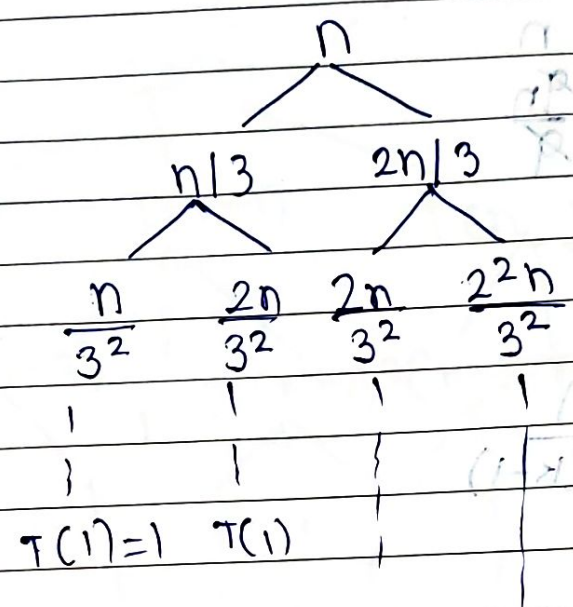
No. of nodes.

Cost

0

1

2



$T(1)=1$ $T(1)$

$T(1)$

$T(1)$

$M + 1 = (n) T$

$$T\left(\left(\frac{2}{3}\right)^k n\right) = T(1)$$

$$(1-x)^n + x^n = 1$$

$$\frac{1}{2} \log \frac{1}{2} + \frac{1}{2} \log \frac{1}{2} =$$

$$\frac{1}{2} \log \frac{1}{2} + \frac{1}{2} \log \frac{1}{2} =$$

$$\frac{2^k}{3^k} n = 1$$

$$n = \left(\frac{3}{2}\right)^k \Rightarrow k = \log_{3/2} n$$

$$\frac{2^2 n}{3^2}$$

no. of nodes	Cost
--------------	------

1 n

$$\frac{2}{n}$$

4 9m

1

1

1

1

$$\frac{n}{n(k-1)}$$

$$T(n) = L + N_c$$

$$2^k + n(k-1)$$

~~$2 \log_3 2n$~~

$$= 2 \log_{3/2} n + n \log_{3/2} n$$

$$= n \log_3 12^2 + n \log_3 12^n$$

$$\leq O(n \log_2 n) \therefore \text{is } \Theta(n \log n)$$

$$T(n) = 2T(n-1) + 1 \quad \begin{matrix} n > 0 \\ n = 0 \end{matrix}$$

Level

0

1

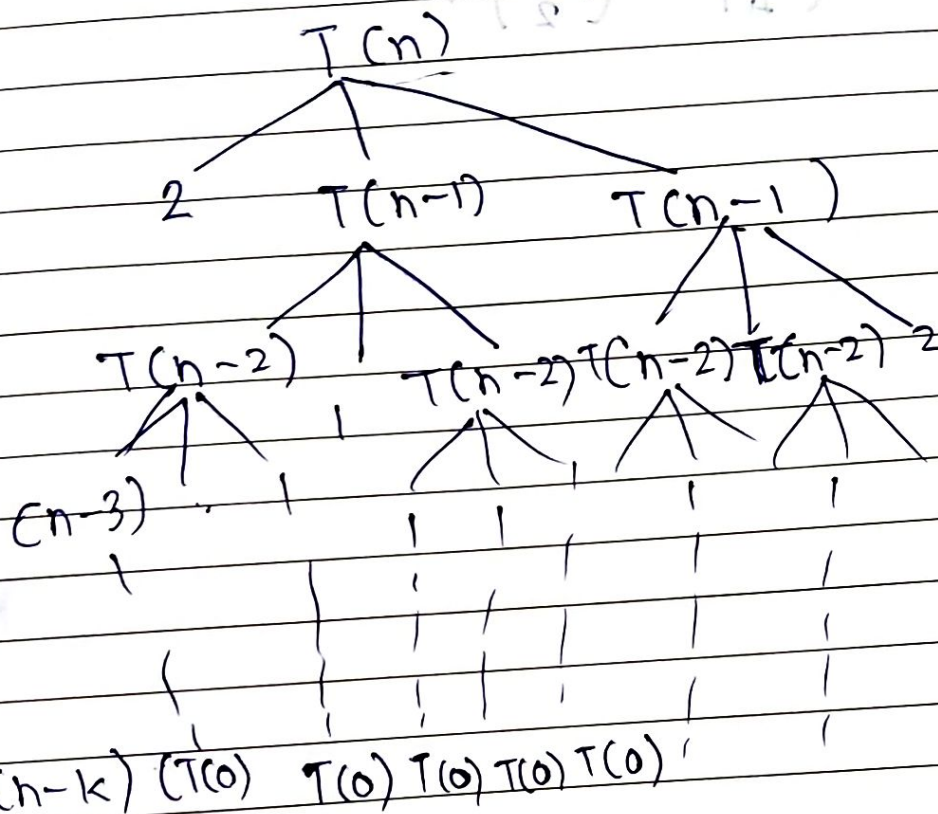
2

3

1

1

k



$$n-k=0$$

$$n=k$$

Total cost:

Cost

1

2

4

8

...

2^k

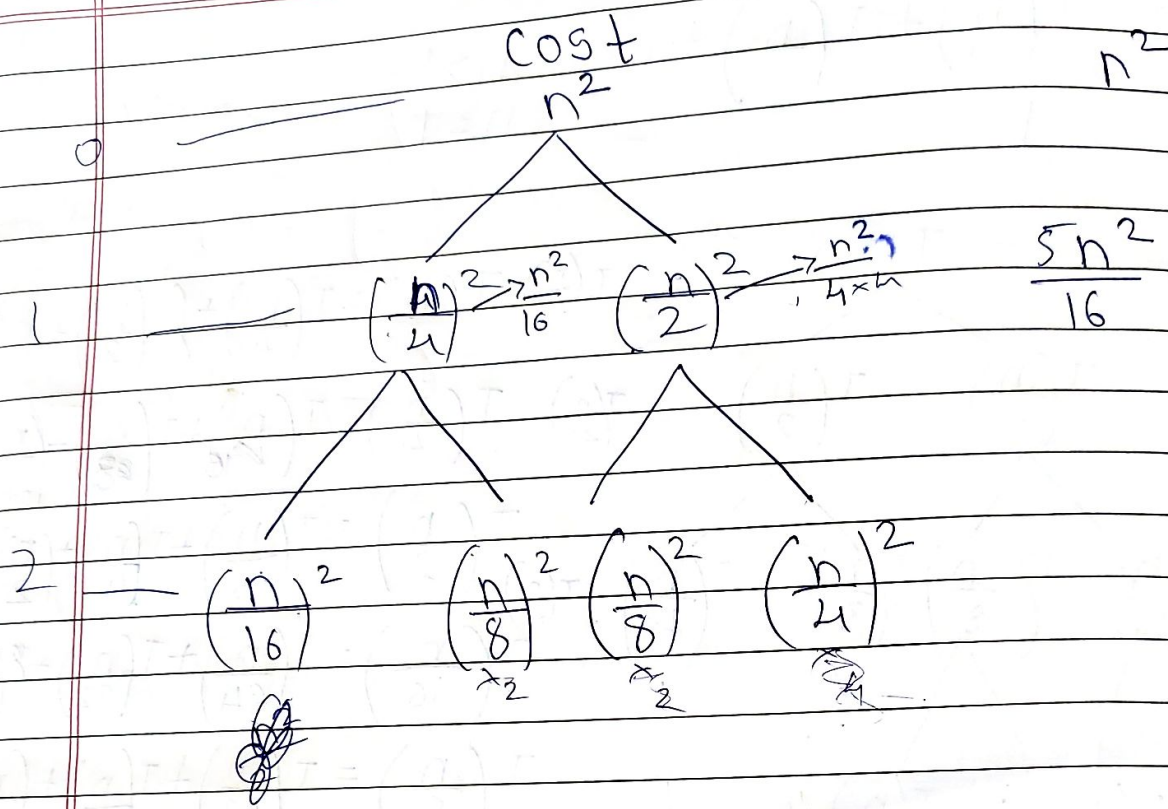
$$T(n) = 1 + 2 + 4 + \dots + 2^k$$

$$T(n) = \frac{2^{n+1} - 1}{2 - 1}$$

$$= 1 \left(\frac{2^{n+1} - 1}{1} \right)$$

$$T(n) = O(2^n)$$

$$TC: 1 + 2 + 4 + 8 + 16 + \dots + 2^k$$



$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{4n}{5}\right) + n \rightarrow \text{Cost}$$

level

0

 $T(n)$

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{4n}{5}\right) + n$$

1 4 $T\left(\frac{n}{5}\right)$ $T\left(\frac{4n}{5}\right)$ 16

$$T\left(\frac{n}{5}\right) = T\left(\frac{n}{5^2}\right) + T\left(\frac{4n}{5^2}\right) + \frac{n}{5}$$

2 $T\left(\frac{n}{25}\right)$ $T\left(\frac{4n}{25}\right)$ $T\left(\frac{4}{25}\right)$ $T\left(\frac{4^2 n}{5^2}\right)$

$$T\left(\frac{4n}{5}\right) = T\left(\frac{4n}{5^2}\right) + T\left(\frac{4^2 n}{5^2}\right) + \frac{4n}{5}$$

$$T\left(\frac{n}{25}\right) = T\left(\frac{n}{5^3}\right) + T\left(\frac{4n}{5^3}\right) + \frac{n}{25}$$

m -- $T(1)$ m' -- -- -- $T(1)$ $T(1)$

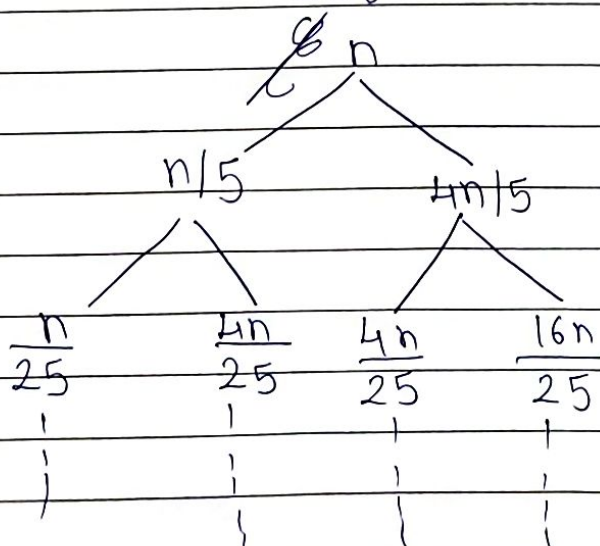
$$T\left(\frac{4n}{25}\right) = T\left(\frac{4n}{5^3}\right) + T\left(\frac{4^2 n}{5^3}\right) + \frac{4n}{5^2}$$

$$T\left(\frac{16n}{25}\right) = T\left(\frac{4^2 n}{5^3}\right) + T\left(\frac{4^3 n}{5^3}\right) + \frac{4^2 n}{5^2}$$

k^{th} -- -- -- $T(1)$

~~Cost~~

cost



1 2 k -- $n(k-1)$

$$k^{\text{th}} \text{ term} = \left(\frac{4}{5}\right)^k n$$

$$1 = \left(\frac{4}{5}\right)^k n$$

$$n = \left(\frac{5}{4}\right)^k$$

$$k = \log_{5/4} n$$

→ leaf node cost

$$\text{Total cost} = n(k-1) + 2k$$

$$= n(\log_{5/4} n - 1) + 2\log_{5/4} n$$



$$\text{Total cost} = 2^k + nk$$

$$= 2^{\log_{5/4} n} + n \cdot \log_{5/4} n$$

$$= n^{\log_{5/4} 2} + n \log_{5/4} n$$

$$= O(n^3)$$